

Cooperative Reinforcement Learning in Euchre: Extending Single-Agent Approaches with QMIX

Devin Abraham, Ben Freidinger, and Holt Klineberg

Rice University / CMOR 544

June 23, 2026

Abstract

Euchre is a cooperative, partially-observable, imperfect information card game, making it a perfect candidate for Reinforcement Learning (RL) methods. Prior research has applied single agent methods including Deep Q-Networks (DQN) and Neural Fictitious Self-Play (NFSP). These methods provided a solid foundation, but they treat teammates as a deterministic part of the environment, rather than a dynamic agent that must be learned to cooperate with. We explored different multi-agent methods, alternating-agent training and QMIX, to better model the cooperative nature of Euchre. In particular, QMIX is a Centralized Training with Decentralized Execution method that combines each agents' local reward into a team reward to encourage cooperation without leaking local observations between agents. After training, the QMIX agents achieved a win rate of over 60% against a rule-based deterministic benchmark, significantly outperforming single-agent methods. Our findings suggest that QMIX and similar multi-agents methods can be effective in training agents for cooperate, imperfect information games.

1 Introduction

Euchre is a two vs two card game popular in the Midwest that the three authors all enjoy playing. Despite Euchre's popularity, it is an under-studied game with only one paper attempting Euchre from a learning perspective. Euchre serves as a challenging game to learn for four main reasons. There is a two vs two structure enforcing a multi agent game with agents trying to cooperate and other's in opposition. There is also partial observability in Euchre each player only knows the cards in their hands and the cards that have been played on the table but not their partners or opponents cards forcing decision under uncertainty. In addition, in Euchre there is a strict no "table talk" rule to explicitly prevent communication between partners which prevents any explicit information passing between agents, but allows for implicit coordination. Finally, there is a massive state space on the order of 10^{25} .

In prior research on Euchre, they treated the game as a single agent, which didn't allow any cooperation between teammates leading to subpar performance against a rule based agent.

- Motivation: card games as benchmarks; Euchre's unique cooperative structure.
- Gap: prior model-free work treats the game as single-agent; ignores the joint-policy nature of partner play.

- Contributions:

1. Reproduction and extension of Pugh’s single-agent DQN baseline.
2. Alternating-agent training as a first cooperative attempt.
3. QMIX (CTDE) implementation for two cooperating Euchre agents.
4. Reward-shaping “personalities” (timid, aggressive, mixed) to probe how shaped incentives affect cooperative play.
5. A revised, lower-variance evaluation protocol (full-game, first-to-10) and head-to-head comparison against Pugh-style baselines.

2 Background

2.1 Euchre

Euchre uses a 24 card deck with (9, 10, J, Q, K, A) of each suit. Players sit in a circle with partners across the table from each other. After dealing 5 card to each player the dealer will flip a card face up which will be a potential trump suit. We now enter the first phase of the game, bidding. Players go one at a time bidding if they want to accept the trump card meaning the suit of the face up card will be the trump and the dealer will get the option to replace the face up trump card with any card in their hand. If all players pass once, then the face up card is flipped over and players are allowed to call any suit to be the trump. If it goes back around to the dealer, they must call. Now the playing phase starts, players will play a trick which consists of all players putting one card down one at a time. Players must follow the lead suit unless they are unable to in which case they may play the trump suit or another suit. The winner of a trick plays the first card of the next trick. The ordering of the trump suit is as follows (J, J of same suit color, A, K, Q, 10, 9). The scoring works as follows if the team that called trump wins 3-4 tricks they get 1 point, if the team that called trump wins 5 tricks they get 2 points (this is called a march), and if the team that didn’t call trump wins 3 or more tricks they get 2 points (this is called getting euchred). The game ends when one team scores 10 points.

2.2 Reinforcement Learning Preliminaries

2.2.1 Decentralized Partially Observable Markov Decision Process (Dec-POMDP)

The Dec-POMDP is defined as a tuple $(\mathbb{D}, \mathbb{S}, \mathbb{A}, T, \mathbb{O}, O, R, h, b_0)$ which each variable is defined as follows.

1. $\mathbb{D} = \{1, \dots, n\}$ is the set of n agents
2. $\mathbb{S}$ is a finite set of states
3. $\mathbb{A}$ is the set of joint actions
4. T is the transition probability function
5. $\mathbb{O}$ is the set of joint observations
6. O is the observation probability function
7. R is the immediate reward function

8. h is the horizon of the problem
9. $b \in \Delta(\mathcal{S})$, is the initial state distribution at time $t = 0$.

2.3 Cooperative Multi-Agent RL [1]

There are three main frameworks for Cooperative Multi-Agent Environments with how the user trains and executes the policies.

2.3.1 Centralized Training and Execution

With this method a centralized controller chooses actions $(a_1, a_2, \dots, a_n)$ for the agents based off of information from each agent and the centralized controller gets each agent's observation $(o_1, o_2, \dots, o_n)$. For Euchre and card games, decentralized execution is necessary to stay within the rules of the game.

2.3.2 Decentralized Training and Execution

With this method each agent observes its current history, takes an action, and makes an observation. Each agent only sees its own history, its own rewards, and the global information. This method is how Eli Pugh trained his agents. In this agents are trained completely on their own making it difficult to gain any cooperation, but are executed in a decentralized manner similar to how Euchre is actually played.

2.3.3 Centralized training for decentralized execution

This method is vaguely defined in the literature but consists of each agent observing their current history, taking an action, and seeing the resulting observation. A centralized planner observes joint information from all agents, providing feedback to the agents about the quality of their actions. After the training phase, agents act in a decentralized manner with access to only their own information and the centralized planner is not used.

This method lends itself well to Euchre allowing the centralized controller to build cooperation between agents during training, while in execution the agents only act on their own information, matching the structure of Euchre.

We considered the following CTDE methods

Value Decomposition Networks (VDN) [2] VDN decomposes the joint Q-function into additive Q-functions per agent, which allows training to occur in a centralized setting, but agents still learn individual Q-functions allowing them to select an action in a decentralized manner. VDN assumes $Q(h, a) \approx \sum_{i \in \mathbb{I}}^n Q_i(h_i, a_i)$. The joint Q-function is a sum of local Q functions. While this method made great strides in MARL, it suffers from severe limitations such as only being able to model linear interactions between agents, and it fails to model complex relations.

QMIX [3] QMIX allows for more general decompositions. QMIX assumes the joint Q-function as a monotonic function of individual Q-functions. $Q(h, a) \approx f_{mono}(Q_1(h_1, a_1), \dots, Q_n(h_n, a_n))$ while enforcing the monotonicity relationship by imposing a constraint $\frac{\partial Q_{tot}}{\partial Q_a} \geq 0$. This property blends very well with CTDE since because of monotonicity agents can greedily select the best action for their own Q-function and take the best action for the joint Q-function.

3 Related Work

3.1 Single-Agent RL on Euchre (Pugh, 2020) [4]

Pugh presents one of the first modern model-free reinforcement learning approaches to Euchre. Since Euchre is a card game where players do not know the other players’ hands, Pugh models the game as a partially observable Markov decision process. Each player can observe their own hand, the flipped card, the played-card history, the current player, and other public information, but the opponents’ hands are hidden. Pugh implements Euchre in **RLCard**, which is an open-source reinforcement learning toolkit for card games. To do this, he defines the Euchre-specific player, game, round, dealer, and judge. He then trains and compares a random agent, a strong hand-coded **RuleAgent**, a DQN agent, and an NFSP agent. The **RuleAgent** is especially important for our project because it was written using human Euchre domain knowledge, so it gives us a strong human-mimicking baseline that we inherit.

For the learning agents, the DQN agent approximates the Q -function with a neural network instead of storing a massive table of values for every state-action pair. It uses standard DQN ideas such as ϵ -greedy exploration and replay memory, following the general idea of Q -learning from Watkins (1989) and deep Q -learning from Mnih et al. (2013). Pugh also trains NFSP agents, which combine a best-response Q network with an average-policy network trained from previous best responses. His main result is that these model-free RL agents learn much better-than-random Euchre strategies, but they do not clearly beat the strong rule-based agents. When evaluated as two trained agents against two **RuleAgents**, the DQN and NFSP agents reach roughly break-even performance, with win rates often around or just above 50%. This result is useful for our project because it shows that single-agent RL can learn competent Euchre play, but it also shows the limitation of this setup. The agents are trained mostly independently, so they are not explicitly learning how to cooperate with their partner. This motivates our use of cooperative multi-agent methods such as CTDE, VDN, and QMIX.

3.2 QMIX: Monotonic Value Factorisation for Deep Multi-Agent Reinforcement Learning (Rashid et al, 2018) [3]

The architecture of QMIX is divided into three main parts: one deep recurrent Q -network (DRQN) per agent, a mixing network that combines the outputs of each DRQN into Q_{tot} , and a set of hypernetworks that generate the weights of the mixing network. Each DRQN takes in the agent’s current observation and last action, and outputs a vector of values for each possible action the agent can take. The hypernetworks take as input the global state (not available to any individual agent) and outputs a set of weights and biases for the mixing neural network. Importantly, all of the weights are run through an absolute value function before being output; this is enough to ensure monotonicity. The mixing network takes in the weights and biases generated by the hypernetworks specific to the global state and each vector output by the individual DRQN’s and outputs Q_{tot} , the overall team reward. The author’s point out the importance of the hypernetworks: the global state is not an input to the mixing network because Q_{tot} does not need to be monotonic in the global state, therefore using the global state to generate weights allows more flexibility.

4 Problem Formulation

4.1 Euchre as a Dec-POMDP

We model the cooperating team as

$$\langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{P}, R, \{\Omega_i\}, \mathcal{O}, \gamma \rangle.$$

The two cooperating agents are $\mathcal{I} = \{0, 2\}$, the two partners seated across the table. The global state $s \in \mathcal{S}$ encodes all four hands, the dealer position, the up-card, the trump suit (once decided), the play history of the current and prior tricks within the hand, and the running score; this state is only available to the mixing network during training. The action space $\mathcal{A}_i$ is stage-dependent: during bidding, agent i chooses among $\{\text{pass}, \text{order up}\}$ in round one and among $\{\text{pass}\} \cup \{\text{call suit } c\}$ in round two, and during play, $\mathcal{A}_i$ is a card from the agent’s hand subject to the follow-suit constraint. Transitions $\mathcal{P}(s' \mid s, a_0, a_2)$ are based on the joint action of the team (a_0, a_2) along with the opponents’ fixed policies and the dealing process; within a hand, transitions are deterministic given the joint action, and only deals are stochastic. The local observation $o_i \in \Omega_i$ contains agent i ’s own hand, the up-card, the trump suit (if set), the public play history, the current score, and whose turn it is, but importantly omits the opponents’ hands and the partner’s hand. The reward R is shared between the two team members and follows the standard per-hand reward from the rules of Euchre: +1 when the team takes three or four tricks as the maker, +2 for a march (all five), −2 when euchred, and +2 when successfully euchring the opposing makers; we also provide a +0.25 reward per trick won. All rewards are matched with a negative penalty applied to the other team to keep the game zero-sum, which can help with training. Finally, we treat each hand as an independent episode and use a discount factor of $\gamma = 0.99$ because episodes are relatively short.

4.2 State and Observation Representation

Each agent has a 48-dimensional local observation vector composed of: binary indicators for seen cards (24), indicators for the current hand (10), trump indicator (1), lead suit indicator (1), inputs to encode the flipped card (2), padding in case you pick up the center card (2), and padding for the up to 4 cards that can appear in the center (8).

The global state is encoded in a 127 dimensional vector: all four hands in binary representation ($4 * 24 = 96$), binary indicator of seen cards (24), trump suit in one-hot encoding (4), current tricks won per team (2), and trick number (1).

Each agent has 54 possible actions: 6 from the bidding phase (pass, dealer pick up, call each of 4 suits), 24 for discarding (if you are the dealer and are told to pick up a sixth card), and 24 for playing cards during a hand.

5 Methods

5.1 Environment

We built off the environments built by Eli Pugh [5] which is itself built on the RLCard open source toolkit [6]. His work defined the state space for a single agent, legal action sets for each state, the game process, and scoring. To implement the QMIX components, we used code from *ElegantRL*, an open source library for deep RL methods, as a model [7].

5.2 Baselines from Pugh

- **RandomAgent** – uniform over legal actions.
- **RuleAgent** – Pugh’s hand-coded heuristic agent; a strong but conservative strategy.
- **Single-Agent DQN** – our reproduction of Pugh’s DQN agent trained for 3 million episodes.

5.3 Alternating Cooperative Training

Our first attempt at cooperative training was using a basic alternating training framework. This is a studied method that can be successful in some cases. The main idea is that we alternate between two phases during training: in phase 1, agent 2’s Q-network is frozen and agent 0 trains playing with the frozen agent 2 (choosing greedily) against two rule based opponents. After a set number of episodes (we used 1500), the roles reverse, agent 0 is frozen and agent 2 trains. This method has an advantage over training agents simultaneously in that at each time, only one agent is training, which provides a more stable environment and target. We initialized both agents as Pugh’s RuleAgent (this was a feature Pugh implemented which performs ”behavioral cloning” to learn the rule-based policy before training truly begins) and trained this model for 3 million episodes (2000 phases).

5.4 QMIX for Euchre

5.4.1 Architecture

Each agent has an independent Q-Network $Q_i(\tau_i, a_i)$ that takes its 48 dimensional state vector and outputs a Q value for each of the 54 possible actions. Illegal actions are preset to $-1e^9$ to disclude them from selection. The network is a multilayer perceptron with two hidden layers (width 128) with tanh activation functions and a linear output layer.

The mixing network takes the two individual Q-values $(Q_0, Q_2) \in \mathbb{R}^2$ and the global state $s \in \mathbb{R}^{127}$ and produces a scalar joint value $Q_{\text{tot}} \in \mathbb{R}$:

$$Q_{\text{tot}}(\tau, \mathbf{a}, s) = \mathbf{w}(s)^\top [Q_0, Q_2] + b(s)$$

where $\mathbf{w}(s) \in \mathbb{R}^2$ and $b(s) \in \mathbb{R}$ are produced by two separate hypernetworks conditioned on s . The hypernetworks are also multilayer perceptrons that take in the 127 dimensional global state vector and output the weights (1) and biases (2) for the mixing network. They each have two hidden layers and use a ReLU, linear, and absolute value activation functions. The weight network outputs two values (one weight per agent), while the bias network outputs a single bias term. The final activation for the weights hypernetwork is the absolute value function, ensuring weights are all positive, which enforces monotonicity.

5.4.2 Training Loop

At each episode, a full hand of Euchre is played. The agents select actions based on ϵ -greedy exploration. Over the course of training, ϵ decays from 0.5 to 0.05 to allow for high exploration at the beginning and more greedy behavior as the agent improves. The two opponents are fixed rule-based agents. Once both agents have chosen actions for a phase (calling trump or a trick), the observation and action information is added to the replay buffer. Rewards are assigned after each phase and shared identically by each teammate according to the values described earlier.

Training begins with 50,000 episodes of warmup where no updates to the networks occur to stabilize the model. After this, one gradient step occurs per episode. We utilized Pugh’s update structure

of using a Double DQN structure, evaluating the actions produced by the online network based on the static target network. The loss is the mean square error between Q_{tot} and the value from the target network. Then, this error is used for the backpropagation step to update the weights backward, starting at the mixing network, then through each individual network.

As in the alternating training case, we initialized the individual networks as rule-based agents using behavioral cloning.

5.4.3 Hyperparameters

Our final set of hyperparameters are summarized in the table below. We were unable to experiment as much as we would have liked with tuning these due to time constraints. The final hyperparameters were chosen by starting with standard values and manually tuning a few.

Table 1: Training hyperparameters.

Hyperparameter	Value
Episodes	5,000,000
Warm-up episodes	50,000
Train every	1 episode
Target network sync	10,000 episodes
Batch size	64
Replay buffer size	500,000
Learning rate	2×10^{-4}
Discount γ	0.99
ϵ range	$0.5 \rightarrow 0.05$
Evaluation interval	100,000 episodes
Games per evaluation	500

6 Experimental Setup

6.1 Evaluation Protocol

In Pugh’s paper, he used straight trick win-rate to quantify success. While this is a solid measure of an agent’s card playing skills, it does not accurately capture its ability to win in Euchre. Furthermore, it is more susceptible to noise, as individual tricks often come down to luck of the deal. We decided to change the evaluation scheme to make it both more true to the structure of the game and less influenced by noise in the evaluation hands. Our scheme involves playing a full game of Euchre: the dealer position rotates around the table and both teams compete to be the first to 10 total points, where the only points allotted are those at the end of a hand. This emphasizes euchres and marches, making the calling phase far more important. For each comparison, we ran 1000 games and calculated 95% CIs to better compare results.

6.2 Opponents

- Random pair.
- Rule-based pair (primary benchmark).
- Pugh-style standalone DQN pair (tests cooperative gain over independent learners).

6.3 Personality Configurations

To probe how reward shaping affects cooperative play, we designed two distinct agent personalities — *timid* and *aggressive* — by modifying the intermediate reward components described in Section 5. The timid personality is built around conservative play: it receives a reduced reward per trick won, incurs a penalty for calling trump, earns no march bonus, faces a steeper euchre penalty, and gains an additional reward for defensive wins. The aggressive personality inverts these incentives, rewarding trick wins more heavily, encouraging trump calls, providing a march bonus, and applying a lighter euchre penalty, with no reward for defensive wins. Table 2 summarizes the reward values for each configuration relative to the standard QMIX model.

Table 2: Reward components for the Timid and Aggressive shaping configurations compared to the standard QMIX model.

Component	Timid	Aggressive	Standard
Trick win	+0.15	+0.35	+0.25
Trump call	−0.2	+0.3	0
March bonus	0.0	+0.5	0
Euchre penalty	−3.0	−1.2	−2.0
Defensive win	+0.3	0.0	0

We trained three pairings of these personalities — timid+timid, aggressive+aggressive, and mixed (aggressive+timid) — each for the full 5M episodes with the same hyperparameters as the standard QMIX model, to test whether complementary roles could synergize and potentially outperform the neutral baseline.

6.4 Compute and Reproducibility

All models were trained using the same hardware, hyperparameters, each running for 5 million episodes, to ensure a fair and consistent comparison across all configurations.

7 Results

7.1 Headline Result

Our QMIX implementation substantially outperforms all single-agent baselines across every opponent configuration: QMIX (basic) achieves a win rate of 79.8% against a random pair, 60.2% against the rule-based pair, and 65.4% against the Pugh-style standalone DQN pair, demonstrating that cooperative CTDE training provides a meaningful and measurable advantage over independent learners even when those learners are trained to competency against the same rule-based opponents.

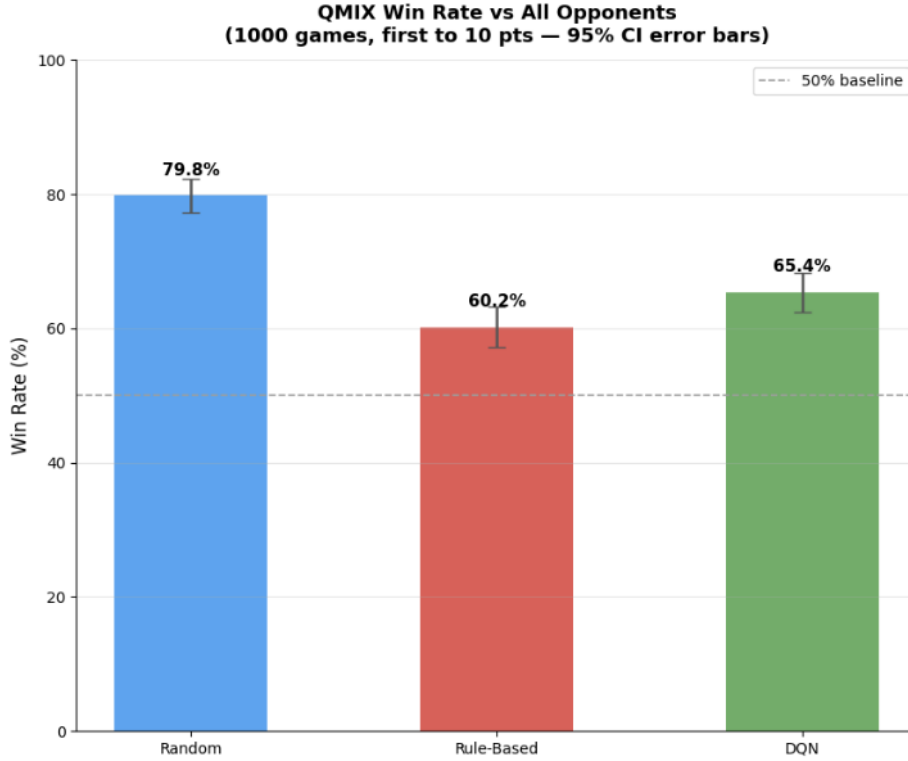
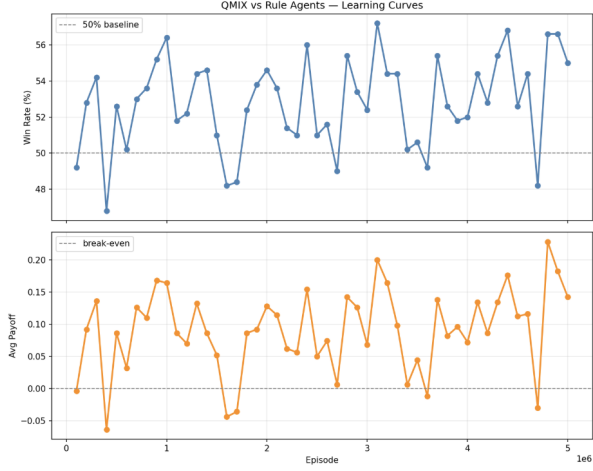


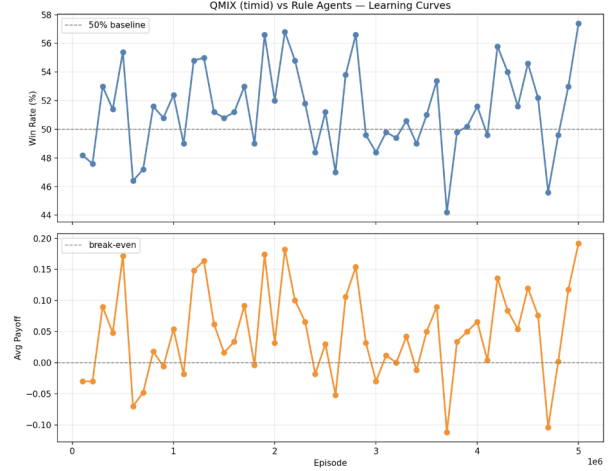
Figure 1: Win rates of QMIX (basic) against each opponent configuration (Random, Rule-based, and Pugh DQN pairs), evaluated over 1000 full first-to-10-point games with 95% confidence intervals.

7.2 Learning Curves

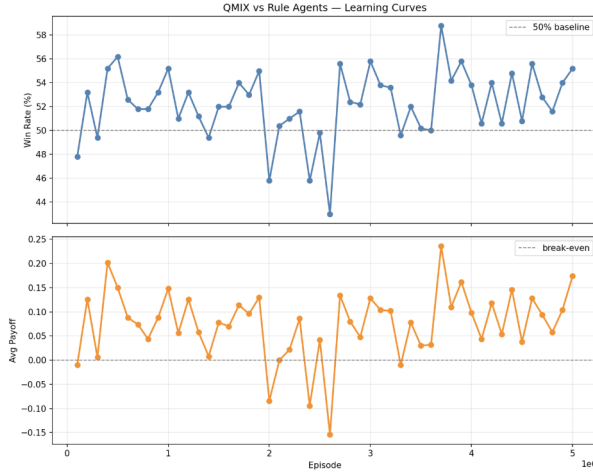
The learning curves for all four QMIX variants; basic, timid, aggressive, and mixed follow a broadly consistent trajectory, shown in Figure 2. Due to behavioral cloning initialization from the rule-based agent, all four models begin training with a non-trivial policy and cross the 50% per-hand win rate threshold within approximately 1 million episodes. Beyond this point, training progresses more gradually, with agents plateauing at per-hand win rates in the range of 52–58% depending on the personality configuration. Importantly, this per-hand advantage translates into a meaningfully larger per-game win rate of 60–64%, as the structure of a first-to-10-points game amplifies small per-hand edges, particularly through marches and euchres, over the course of a full game.



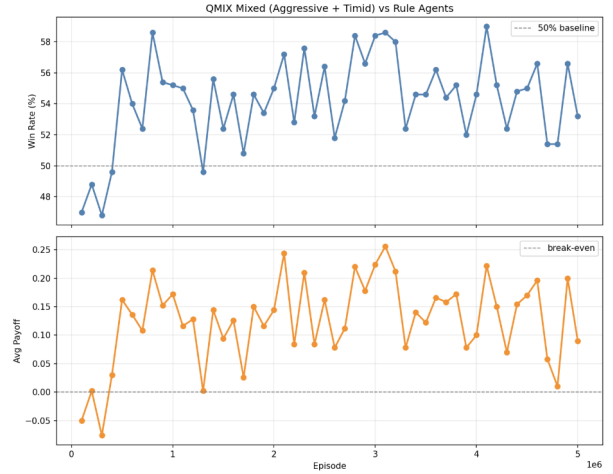
(a) Basic QMIX



(b) Timid



(c) Aggressive



(d) Mixed (Aggressive + Timid)

Figure 2: Per-hand win rate against the rule-based opponent over 5M training episodes for each QMIX personality variant. The dashed line marks the 50% win rate threshold. All variants are initialized via behavioral cloning from the rule-based agent and cross 50% within approximately 1M episodes, plateauing at 52–58% per-hand win rate.

7.3 Personality Comparison

We tested three combinations of reward-shaped personality agents; timid+timid, aggressive+aggressive, and mixed (aggressive+timid) alongside the standard QMIX model. Results are shown in Figure 3. The full-game win rates follow a consistent ordering: timid 60.2%, aggressive 62.2%, and mixed 63.8%, with corresponding average score margins of +1.1, +1.3, and +1.4 points respectively. The mixed pairing achieves the best overall performance, suggesting that asymmetric role specialization, one caller-aggressive agent paired with a conservative partner, captures the upside of aggressive trump calling while limiting exposure to costly euchres.

Crucially, the personalities differ most sharply at the hand-event level rather than at the trick-taking level. Trump call rate, march rate, euchre rate, and “got euchred” rate all vary meaningfully across personality types, while per-hand win rate remains roughly flat at approximately 53% across all configurations. This indicates that the personality-driven gains in full-game win rate stem

primarily from agents calling trump, not from any improvement in the card-playing phase itself.

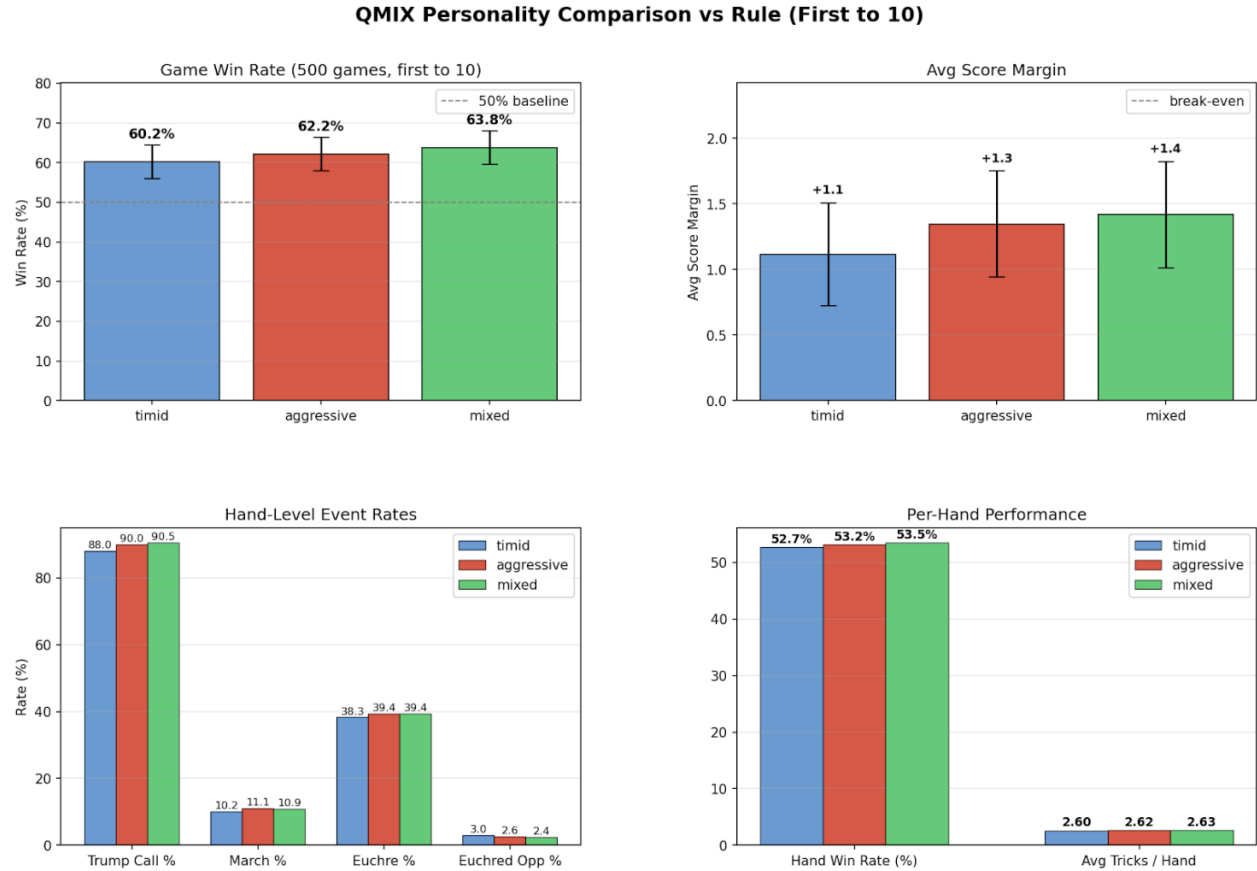


Figure 3: Comparison of reward-shaped personality configurations (timid+timid, aggressive+aggressive, mixed) and the standard QMIX model. Panels show full-game win rate, average score margin, per-hand win rate, and hand-level event rates (trump call %, march %, euchre %, got euchred %).

7.4 Hand-Outcome Breakdown

Figure 4 breaks down the distribution of hand outcomes for QMIX (basic) against the rule-based opponent. The plurality of hands fall into the “QMIX called and won” category at 53%, meaning the QMIX team successfully called trump and secured at least three tricks. Meanwhile, instances of “got marched”, where the opposing rule-based team swept all five tricks remain rare. This pattern confirms that QMIX’s edge over the rule-based benchmark is driven primarily by more confident and accurate trump-calling decisions during the bidding phase, rather than superior trick-taking play once a hand is underway.

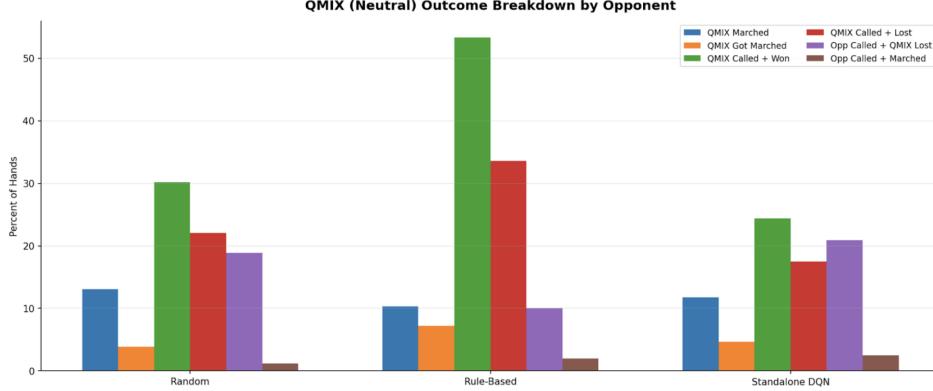


Figure 4: Distribution of hand outcomes for QMIX (basic) vs. the rule-based opponent pair over 1000 full games, broken down by outcome category.

7.5 Alternating Training Failure

Our alternating cooperative training approach failed to produce competitive agents, achieving only approximately 46% win rate against the rule-based pair, falling below even the single-agent DQN baseline and indicating that naive alternating optimization is an insufficient strategy for learning cooperation in Euchre.

We attribute this failure primarily to the non-stationarity introduced by the alternating training scheme. When agent 0 trains while agent 2 is frozen, agent 0 adapts its policy to best respond to a fixed partner. When the roles then reverse, agent 2 optimizes against a now-frozen but recently updated agent 0. This creates a slow, cycling dynamic in which each agent perpetually chases the other’s most recent policy rather than converging to a stable joint policy. In the worst case, this degrades into a local best-response cycle in which both agents settle into a mutually suboptimal equilibrium. Behavioral cloning initialization partially mitigates this instability by providing a strong starting point, but it does not resolve the fundamental non-stationarity of the alternating update scheme.

7.6 Comparison with Pugh’s Reported Numbers

Table 3 provides a direct apples-to-apples comparison of per-hand win rates against the rule-based opponent across all relevant methods. Our DQN reproduction closely matches Pugh’s originally reported figures, validating our environment implementation. The QMIX models show a clear and consistent improvement over the DQN baselines at the per-hand level, with the gap widening further at the full-game level.

Table 3: Per-hand win rate vs. the rule-based opponent for each method, with 95% confidence intervals. Pugh’s numbers are taken directly from [4]; all others are evaluated under our first-to-10-points protocol.

Method	Win Rate (%)	95% CI
Pugh’s DQN (reported)	~ 50.0	–
Pugh’s NFSP (reported)	~ 50.0	–
Our DQN reproduction	43.6	[40.5, 46.7]
QMIX (Neutral)	54.0	[50.9, 57.1]
QMIX (Aggressive)	55.6	[52.5, 58.7]
QMIX (Mixed)	51.3	[48.2, 54.4]
QMIX (Timid)	52.6	[49.5, 55.7]

We note that Pugh’s evaluation used straight trick win rate, which is more susceptible to noise from individual deals and does not capture the strategic importance of the calling phase. Additionally Pugh never reports his explicit win rate, just as ”over 50%”. Our first-to-10-points protocol better reflects true Euchre competence and amplifies the cooperative advantage that QMIX gains through coordinated trump calling, so readers should interpret cross-protocol comparisons with this caveat in mind.

8 Discussion

8.1 Why CTDE Helps Here

Independent learners suffer from non-stationarity: as each agent updates its policy, the environment shifts from its teammate’s perspective, making convergence slow and cooperation difficult to achieve. QMIX resolves this by computing a joint value Q_{tot} during training that conditions on the global state and both agents’ actions simultaneously, providing a stationary credit-assignment signal. The monotonicity constraint $\frac{\partial Q_{\text{tot}}}{\partial Q_i} \geq 0$ ensures this centralized signal remains consistent with decentralized greedy execution, allowing the cooperative policy learned during training to transfer cleanly to gameplay.

8.2 What the Personalities Tell Us

The consistent ordering mixed > aggressive > timid suggests that asymmetric role specialization is beneficial in Euchre. Timid agents are too reluctant to call trump, ceding control of the hand, while purely aggressive agents overcommit and get euchred. The mixed pairing captures the upside of aggressive trump-calling while the conservative partner limits tail-risk hands — a division of labor that mirrors sound human Euchre strategy and emerges here from reward shaping alone, without any explicit coordination mechanism.

8.3 Limitations

- The rule-based opponent is a ceiling we can compete well against but not decisively beat, probably because expert human play is itself largely rule-based.
- Behavioral cloning initialization may bias the joint policy search toward the rule agent’s strategy, limiting exploration of novel cooperative behaviors.

- Only one team is learned; fixed opponents do not adapt, limiting robustness against varied strategies.

9 Future Work

We have a few ideas for possible extensions to our model. First, there are more complex cooperative frameworks QPLEX and QTRAN that build off QMIX but allow for even greater flexibility in the relationship between individual agent networks and the mixer. We also are considering the potential to merge QMIX and Monte Carlo Tree Search to create a hybrid model with error that is easier to bound (thanks to Professor Hengrui Luo for the idea). A final extension involves the plethora of "house rules" that exist in Euchre. It is common in practice to have additional rules in place on top of the standard rules that allows players to take additional actions in particular scenarios. We could add in a few of these to the model to see if QMIX can deal with the added complexity.

10 Conclusion

In this study, we demonstrated that the application of cooperative Centralized Training with Decentralized Execution (CTDE) via QMIX materially improves agent performance in Euchre compared to traditional single-agent methods. By explicitly modeling the strategic relationship between teammates rather than treating partners as static environment variables, our QMIX agents achieved a game win rate exceeding 60% against a robust rule-based benchmark. While our exploration into agent personalities revealed that specific pairings can further amplify team success, the core value-decomposition framework proved essential for navigating the game's imperfect information and cooperative requirements. These findings provide a strong foundation for future research into multi-agent coordination within complex, trick-taking card games and underscore the superiority of cooperative MARL approaches over independent learners.

References

- [1] Christopher Amato. 2024. *A First Introduction to Cooperative Multi-Agent Reinforcement Learning*.
- [2] Peter Sunehag et al. 2017. Value-Decomposition Networks for Cooperative Multi-Agent Learning. *arXiv:1706.05296*.
- [3] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. 2018. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. In *ICML*.
- [4] Eli Pugh. 2020. Learning to Play Euchre with Model-Free Reinforcement Learning. Stanford.
- [5] Eli Pugh. 2021. *Euchre: Deep Q Learning for Euchre*. GitHub repository. <https://github.com/elipugh/euchre>.
- [6] Daochen Zha, Kwei-Herng Lai, Songyi Huang, Yuanpu Cao, Keerthana Gopalakrishnan, Mehmet Erdem Demir, Juyoung Suh, Ruzhe Wei, Junyu Guo, and Xia Hu. 2020. *RLCard: A Toolkit for Reinforcement Learning in Card Games*. arXiv preprint arXiv:1910.04376.

- [7] Xiao-Yang Liu, Zechu Li, Ming Zhu, Zhaoran Wang, and Jiahao Zheng. 2021. *ElegantRL: Massively Parallel Framework for Cloud-native Deep Reinforcement Learning*. GitHub repository. <https://github.com/AI4Finance-Foundation/ElegantRL>.
- [8] Xiao-Yang Liu, Zechu Li, Zhuoran Yang, Jiahao Zheng, Zhaoran Wang, Anwar Walid, Jian Guo, and Michael I. Jordan. 2021. ElegantRL-Podracers: Scalable and Elastic Library for Cloud-native Deep Reinforcement Learning. In *NeurIPS Workshop on Deep Reinforcement Learning*.